Hafta 2 — Slayt açıklamaları ve sunum metni

Sunum: CPU yönetimi ve süreç zamanlama — Hafta 2: C prototipi ve ilk deneyler. Yürütücü: Burak Şanlı. Danışman: Prof. Dr. Dursun Akaslan.

Metin, 18 slaytlık PDF'in sırasını ve ölçümlerini izler. Ana bölüm konuşma dilindedir. Canlı gösterim ve soru-cevap notları ayrı bölümlerdir. Okuma hızına göre yaklaşık 20 dakikalık anlatım öngörülebilir; canlı gösterim ek süre gerektirir. Sayısal sonuçlar 5 Kasım 2026 arşivine aittir; yeni yerel ölçümler ayrı deney olarak değerlendirilir.

Slayt 1 — CPU yönetimi ve süreç zamanlama

Merhaba, ben Burak Şanlı. Bitirme projemde CPU yönetimi içinde süreç zamanlama konusuna odaklanıyorum. Danışmanım Prof. Dr. Dursun Akaslan. Bu sunumda ikinci haftada ortaya çıkan C prototipini, ölçüm altyapısını ve ilk deney sonuçlarını anlatacağım.

Araştırmanın temel amacı, uzun arka plan işleriyle birlikte çalışan kısa ve etkileşimli görevlerin ilk çalışma fırsatını iyileştirebilecek bir politika araştırmak. Bunu yaparken uzun işlerin ilerlemesini de korumak gerekiyor. Bu hafta iki ayrı araç hazırlandı: kuralları inceleyen zamanlama modeli ve mevcut Linux davranışını ölçen gerçek süreç programı. Özel politika henüz Linux çekirdeğine yüklenmedi. Anlatım boyunca model sonucu ile gerçek Linux ölçümünü birbirinden ayıracağım.

Slayt 2 — Araştırma sorusu ve bu haftanın kapsamı

Ele aldığım senaryoda derleme veya uzun hesaplama işleri CPU'yu yoğun biçimde kullanırken kısa görevler de çalışmaya hazır oluyor. Araştırma sorum şu: kısa görevlerin beklemesini, uzun işlerin ilerlemesi korunarak azaltabilir miyiz? İlk hafta bu problem ve teknik seçenekler tanımlanmıştı. İkinci haftada fikri çalıştırılabilir kod, test ve deney kayıtlarına dönüştürdük.

Önce davranışını izleyebildiğimiz bir C modeli kurduk. Model, görevlerin varış anını ve CPU ihtiyacını girdi olarak alıyor. Aynı görevleri iki politikada çalıştırarak farkı görebiliyoruz. Gerçek Linux programı ise mevcut sistemin başlangıç davranışını kaydediyor. Çalışma, bütün çekirdeği yeniden yazmak yerine CPU zamanlamasında sınırları belli bir bileşene odaklanıyor. Ayrıca her uygulama yavaşlamasını CPU'ya bağlamıyorum; bellek, disk ve sanallaştırma da etkili olabilir.

Slayt 3 — Geliştirilen bileşenler

İki C dosyasının görevleri farklı. scheduler_sim.c tek CPU üzerinde görev seçimini modelliyor; Round Robin referansını ve bütçe/yaşlandırma kullanan prototipi karşılaştırıyor. Buradaki görevler, veri yapılarında temsil edilen model görevleri. linux_baseline.c ise gerçek Linux süreçleri oluşturuyor, aynı izinli CPU'ya sabitliyor ve uyanma gecikmeleriyle işçi sürelerini ölçüyor. Mevcut zamanlayıcıyı değiştirmiyor.

Python test dosyası, bilinen örnekleri ve çıktı tutarlılığını kontrol ediyor. Deney betiği testleri, model koşularını ve gerçek ölçümleri belirli bir sırayla çalıştırıyor. Arşivde ham CSV/JSON dosyaları, test günlüğü, ortam bilgileri ve kaynak özetleri bulunuyor. Böylece bir grafikteki sayının hangi deneyden üretildiğini geriye doğru inceleyebiliyoruz. GitHub da kaynakların ve belgelerin zaman içindeki değişimini görünür kılıyor.

Slayt 4 — Derleme ve deneylerin tekrarlanması

Tam iş akışı Linux, C11 destekli GCC, GNU Make ve Python 3.10 veya üzerinde çalışıyor. Windows'ta kaynaklar incelenebiliyor ve taşınabilir model derlenebiliyor; gerçek Linux ölçümü için WSL gibi bir Linux ortamı gerekiyor.

Ekrandaki üç komutun sorumlulukları farklı: make iki programı derliyor, make test otomatik kontrolleri çalıştırıyor, make experiment ise testlerle birlikte deney grubunu yürütüyor. Yeni sonuçlar ayrı bir dizine yazılıyor. Son başarılı deneyin konumu latest.json içinde tutuluyor; başarısız bir koşu bu işaretçiyi değiştirmiyor.

Aynı model girdileri ve parametreleri deterministik sonuç verir. Gerçek Linux ölçümlerinin ise sayısal olarak aynı çıkması beklenmez; diğer programlar, donanım ve sanallaştırma etkileri değişebilir. Bu nedenle sunumdaki tarihsel veriyi göstermekle yeni bir canlı ölçüm almak farklı işlemlerdir.

Slayt 5 — Görev girdisi ve süre ölçütleri

Girdideki her satır bir model görevi. id benzersiz görev numarası; class sınıfı; arrival_ms hazır olma anı; burst_ms toplam CPU ihtiyacı. Buradaki görev numarası Linux süreç kimliği değil. Sınıfı da işletim sistemi keşfetmiyor; dosyada açıkça belirtiyoruz.

İlk yanıt, ilk başlama eksi varış. Dönüş süresi, tamamlanma eksi varış. Toplam bekleme ise dönüş süresinden CPU ihtiyacını çıkararak hesaplanıyor. Modelde I/O olmadığı için bu değer bütün kuyruk beklemelerinin toplamını veriyor.

Örneğin 5. milisaniyede gelen görev ilk kez 16'da başlarsa ilk yanıtı 11 milisaniye. Aynı görev 8'de başlarsa 3 milisaniye. Fakat ilk kez erken çalışma, sonraki zaman dilimlerinde az bekleme anlamına gelmiyor. Bu nedenle ilk yanıt, toplam bekleme ve tamamlanmayı ayrı ölçüyorum.

Slayt 6 — Politikanın başlangıç kuralları

Zaman dilimi 4 milisaniye. Görev tamamlanmazsa dilim sonunda yeniden kuyruğa giriyor. Politika, sabit 20 milisaniyelik pencerede 8 milisaniyelik etkileşimli tercih bütçesi kullanıyor. Bütçe pozitifken etkileşimli sınıf tercih ediliyor; bütçe bitince hazır arka plan görevlerine sıra veriliyor.

Yaşlandırma kuralı, seçim anında en az 20 milisaniye beklemiş görevleri öne alıyor. Her iki sınıfa uygulanıyor; en eski bekleme başlangıcına sahip görev seçiliyor. Sınıf içinde kuyruk sırası korunuyor. Hazır iş varken CPU boş bırakılmıyor.

Bütçe katı bir CPU kotası değil: başka sınıfta hazır iş olmaması veya yaşlandırma kararı istisna oluşturabiliyor. Kullanılmayan bütçe sonraki pencereye taşınmıyor. Bu parametreleri en iyi değerler olarak sunmuyorum; ilk davranışı sınamak için belirlenmiş başlangıç değerleri olarak kullanıyorum.

Slayt 7 — Bir model adımındaki karar sırası

Model tam sayı milisaniyelerle ilerliyor. Önce pencere sınırında bütçe yenileniyor, yeni gelen görevler kabul ediliyor. Çalışan görevin zaman dilimi bitmişse veya bütçe nedeniyle yeniden seçim gerekiyorsa görev kuyruğun sonuna alınıyor. Seçimde önce yaşlandırma, ardından bütçeye göre sınıf ve kuyruk sırası değerlendiriliyor.

Seçilen görev bir milisaniye model işi yapıyor; kalan ihtiyaç, zaman dilimi ve bütçe güncelleniyor. İhtiyaç sıfıra inerse bitiş anı yazılıyor. Hazır görev yoksa zaman ilerliyor. Modelin bir milisaniyesi, bilgisayarın gerçekten o süre beklemesi anlamına gelmiyor.

Yaşlandırma ilk varıştan beri geçen toplam zamana değil, son yeniden kuyruğa girişten beri beklemeye dayanıyor. Yeni görev gelişi veya pencere yenilenmesi tek başına kesme oluşturmuyor. Sıfır bütçeyle yaşlandırmadan seçilen etkileşimli görev bir milisaniye çalışabilir; arka plan hazırsa sonraki adımda tekrar seçim olur. Bu ayrıntılar politikanın gerçek sınırlarını belirliyor.

Slayt 8 — Kontrollü örnekte ilk yanıt

İlk iki görev başlangıçta geliyor ve yirmişer milisaniye CPU istiyor. Üçüncü görev birinci milisaniyede geliyor, yalnız bir milisaniye CPU istiyor. Round Robin'de ilk görev sıfır–dört, ikinci görev dört–sekiz arasında çalışıyor. Kısa görev sekizinci milisaniyede başlayabiliyor. Birinci milisaniyede geldiği için ilk yanıtı yedi.

Prototipte ilk dilim bitince etkileşimli üçüncü görev seçiliyor ve dört–beş arasında tamamlanıyor. İlk yanıtı dört eksi bir, yani üç milisaniye oluyor. Ardından uzun işlere sıra geliyor. Bu küçük örnek otomatik testte de kontrol ediliyor.

Örnek, tercih kuralının anlaşılır bir durumda nasıl çalıştığını gösteriyor; bütün yüklerde iyi olacağını kanıtlamıyor. Tabloda yalnız ilk dokuz milisaniye var; bütün işlerin tamamlanmasını ayrıca izlemeliyiz. Bu zamanlar gerçek Linux ölçümüne değil, modele ait.

Slayt 9 — Karşılaştırılan model senaryoları

Üç senaryomuz var. cpu_only girdisinde üç görevin her biri 120 milisaniye CPU istiyor; toplam 360. mixed girdisinde bunlara sekiz tane ikişer milisaniyelik etkileşimli görev ekleniyor; toplam 376. interactive_pressure girdisinde üç arka plan görevi seksener, on iki etkileşimli görev yirmişer milisaniye istiyor; toplam iş 480 milisaniye.

Son senaryo, etkileşimli talebin arttığı koşulda tasarımın sınırlarını incelemeye yarıyor. Görevlerin varış anları CSV dosyalarında kayıtlı. Her iki politika aynı görev kümesini kullanıyor; karşılaştırmada girdiyi değiştirmiyoruz.

Model tek CPU'lu; I/O, CPU göçü ve bağlam değiştirme maliyeti yok. Round Robin anlaşılır bir referans; Linux'un adil zamanlayıcısının modeli değil. Dolayısıyla bu karşılaştırma doğrudan Linux'a karşı hızlanma oranı vermez.

Slayt 10 — Karışık model iş yükünün sonucu

Etkileşimli ortalama ilk yanıt Round Robin'de 11, prototipte 3 milisaniye. Arka plan görevlerinin ortalama dönüş süresi her ikisinde 372 milisaniye. Son görevin tamamlandığı an da iki politikada 376.

Bu örnekte kısa görevler daha erken çalışabiliyor; arka planın ortalama tamamlanma ölçütü değişmiyor. Ancak bütün arka plan ölçütleri aynı değil: üçüncü arka plan görevinin ilk başlama anı 8'den 10'a kayıyor. İlk başlama ve tamamlanma farklı ölçütler.

Son bitişin aynı kalması, aynı CPU işinin modelde boşluk ve bağlam değiştirme maliyeti olmadan yürütülmesiyle uyumlu. Bu bir gerçek sistem işlem hacmi kazancı anlamına gelmiyor. Canlı gösterimde etkileşimli satırların response_ms sütununu ve bütün görevlerin finish_ms değerlerini karşılaştırarak sonucu gösterebilirim.

Slayt 11 — Yoğun yükte ölçütler farklılaşabiliyor

Yoğun etkileşimli yükte ortalama ilk yanıt yaklaşık 31,333'ten 19,333 milisaniyeye düşüyor. Ancak ortalama toplam bekleme yaklaşık 252,667'den 303,833 milisaniyeye yükseliyor. Yalnız ilk grafiğe bakmak bu gerilemeyi gizler.

Bir göreve ilk kez erken CPU vermek, kalan işini de erken bitirmekle aynı şey değil. Sonraki zaman dilimlerinde daha fazla bekleme oluşabiliyor. Bu nedenle ilk yanıttaki kazancı bütün sistemin iyileşmesi olarak sunmuyorum.

Bu karşı örnek araştırmanın yönünü belirliyor: bütçe, yaşlandırma ve zaman dilimi kurallarını farklı yüklerde birlikte değerlendirmek gerekiyor. Hem uzun işlerin ilerlemesini hem kısa işlerin tamamlanmasını inceleyeceğim. Olumsuz koşulları raporlamak, politikanın hangi durumda uygun olmadığını da ortaya koyuyor.

Slayt 12 — Gerçek Linux ölçümünün düzeni

Şimdi modelden gerçek süreç ölçümüne geçiyoruz. Program, ana süreçle sıfır veya iki arka plan işçisini aynı izinli mantıksal CPU'ya sabitliyor. Aynı CPU'da rekabet oluşturuyoruz, fakat CPU'yu diğer sistem işlerinden bütünüyle yalıtmıyoruz.

Ana süreç 40 periyodik kısa CPU patlaması yapıyor. Bunlar 40 ayrı süreç değil; tek sürecin tekrarlanan işleri. Periyot 20 milisaniye, kısa CPU hedefi 500 mikrosaniye. Her arka plan işçisinin CPU bütçesi 800 milisaniye.

Uyanma hedefleri monoton saatte mutlak olarak belirleniyor. İş bütçesi süreç CPU saatiyle ölçülüyor. Kuyrukta beklemek CPU bütçesini aynı şekilde tüketmiyor, fakat duvar süresini uzatıyor. Üç tekrarda önce sıfır, sonra iki işçili koşul var: altı koşu ve toplam 240 örnek. Özel politika yüklenmiyor; normal SCHED_OTHER davranışı kaydediliyor.

Slayt 13 — Gerçek Linux: P95 uyanma gecikmesi

P95 için bir koşunun 40 örneğini küçükten büyüğe sıralıyoruz. En yakın sıra yönteminde 0,95 çarpı 40, yani 38. sıradaki örnek seçiliyor. Bu dağılımın özetidir; kesin bir en kötü durum garantisi değildir.

İşçi olmayan üç koşuda P95 yaklaşık 0,289, 0,129 ve 0,166 milisaniye. İki işçi varken yaklaşık 2,821, 2,969 ve 2,772 milisaniye. Bu arşivlenmiş veri grubunda arka plan yüküyle daha yüksek uyanma gecikmesi gözlenmiş.

Bunu özel politikanın Linux'taki kazancı olarak yorumlamıyorum; iki koşulda da mevcut Linux davranışı ölçüldü. Timer, kesmeler ve sanallaştırma da ölçüme katılabilir. Aynı programı kendi bilgisayarımda çalıştırdığımda sayılar farklı çıkabilir. PDF tarihsel deneyi gösteriyor; yeni ölçümler ayrı kaydedilmelidir.

Slayt 14 — Arka plan işlerinin tamamlanması

Her işçi 800 milisaniye CPU zamanı hedefliyor. Aynı CPU'yu paylaştıkları için duvar süreleri yaklaşık 1620–1625 milisaniye arasında. Bütün işçiler tamamlanmış. CPU'da harcanan süre ile beklemeyi de içeren duvar süresi farklı şeyler.

İşçi süresi, ana programın bekleme çağrısından döndüğü ana göre hesaplanmıyor. Her işçi hesaplamaya başlarken ve bitirirken kendi ölçümünü alıyor. Ana süreç de hepsinin başarıyla bittiğini kontrol ediyor. Böylece bir işçinin yarım kalması başarı gibi gösterilmiyor.

Bu iş yükü sabit sayıda hesaplama işlemi yapmak yerine süreç CPU zamanı bütçesiyle sınırlı. İşlemci hızına göre yapılan döngü sayısı değişebilir. Dolayısıyla bu tablo, sabit bir işin daha hızlı tamamlandığını gösteren bir hızlanma testi değil. İşlem hacmi için sabit iş miktarlı ayrı deney gerekli.

Slayt 15 — Otomatik doğrulamanın sonucu

Kayıtlı koşuda 13 test yöntemi başarılı; başarısız veya atlanan yok. Bu sayı üst seviye test yöntemlerini ifade ediyor; bazıları birçok girdi için alt kontroller yürütüyor.

Analitik iki görevli Round Robin takvimi, geç gelen tek görev, girdi sırası, etkileşimli tercih, bütçe ve yaşlandırma örnekleri kontrol ediliyor. Sabit tohumlu 20 iş yükü iki politikada da çalıştırılıp görev bitişleri ve süre hesapları doğrulanıyor. Bozuk girdilerin ve geçersiz seçeneklerin reddi sınanıyor.

Gerçek Linux testleri sıfır ve iki işçiyle küçük koşuların bitmesini, örnek sayılarını ve sürelerin tutarlılığını denetliyor. Belirli bir makinede belirli gecikmenin mutlaka sağlanmasını istemiyor. Bu testler seçilen örneklerde güven sağlıyor; tüm yüklerde adalet veya açlık oluşmaması ispatı vermiyor. Çekirdek politikasının yüklenmesi de henüz doğruladıkları bir özellik değil.

Slayt 16 — Sınırlılıklar ve yorum sınırı

Model tek CPU ve tek patlamalı görevlerle sınırlı. I/O, uyku/uyanma döngüleri, çok çekirdek yerleşimi ve bağlam değiştirme maliyeti yok. Sınıflar açıkça giriliyor; otomatik sınıflandırma uygulanmadı. Bütçe ve yaşlandırma da kesin gecikme üst sınırı sağlamıyor.

Gerçek ölçümde yalnız üç tekrar var; koşul sırası sabit ve ana sistemin bütün yükü kontrol edilemiyor. Bu nedenle sonuçların genellenmesi ve yalnızca zamanlayıcıya ait nedensellik kurulması sınırlı.

Özel politika henüz Linux çekirdeğinde çalışmıyor. Bu verilerden Linux'a karşı performans artışı çıkarılamaz. Yerel Ubuntu ortamı da ilk Debian deneyinin birebir doğruladığı bir ortam değil. Bu sınırları açıklamak, hangi adımı bitirdiğimizi ve sırada hangi doğrulamaların olduğunu netleştiriyor.

Slayt 17 — İkinci haftanın somut çıktıları

Bu hafta fikir, çalıştırılabilir ve test edilebilir ilk uygulamaya dönüştü. C politika modeli, gerçek Linux başlangıç ölçüm programı, testler, üç senaryo ve deney betiği hazır. Altı model ve altı gerçek Linux koşusu arşivlendi.

Test günlüğü, kaynak özetleri, ortam ve parametre bilgileri korunuyor. Deney grafikleri arşivdeki verilerden, küçük kontrollü zaman çizelgesi otomatik testteki görev tanımından geliyor. Böylece her sayının dayanağı izlenebiliyor.

Teknik belgeler ve sunum GitHub'da sürümlendiriliyor. İkinci haftanın çıktısı tamamlanmış bir çekirdek zamanlayıcısı değil; kuralları denetlenebilir hâle getiren bir prototip ve gerçek entegrasyon için ölçüm temeli. Gelişimi sonraki haftalarda aynı kayıt düzeniyle takip edeceğiz.

Slayt 18 — Sonraki aşama ve geçiş ölçütleri

Sonraki teknik adım hedef çekirdeği ve ayrı test sanal makinesini belirlemek. sched_ext, BPF/BTF, derleyici araç zinciri ve yükleme yetkileri ayrı ayrı doğrulanmalı. Yeni sürüm numarası tek başına gerekli özelliklerin hazır olduğunu göstermiyor.

Önce küçük bir örnek politika derlenip yüklenmeli, etkinliği gözlenmeli ve kaldırılmalı. Varsayılana dönüldüğünde işlerin ilerlediği doğrulanmalı. Bu yol sağlandıktan sonra bütçe ve yaşlandırma gerçek çekirdek altyapısına aktarılacak.

Varsayılan ve özel politika aynı gerçek yüklerle karşılaştırılacak. Daha çok tekrar, koşul sırasının kontrolü, uzun işlerin ilerlemesi ve politika maliyeti birlikte değerlendirilecek. Araştırmanın amacı her durumda daha hızlı olduğumuzu söylemek değil; hangi yükte hangi dengenin oluştuğunu kod ve ölçümle açıklamak. Dinlediğiniz için teşekkür ederim.

Canlı gösterim sırasında anlatım

Komutlar: Çalıştırma ve canlı gösterim.

Derleme ve test

“Kaynakları derliyorum ve test paketini çalıştırıyorum. Test sayısı ve başarı satırı ekranda. Bu testler seçilen örneklerin doğruluğunu gösteriyor; bütün yükler için performans veya adalet garantisi vermiyor.”

Karışık model

“Önce görev dosyasını, ardından aynı girdiyle iki politikayı gösteriyorum. Beşinci milisaniyede gelen kısa görev RR'de 16'da, prototipte 8'de başlıyor. İlk yanıt sırasıyla 11 ve 3. Son bitiş her ikisinde 376. Bunlar model zamanları; terminal komutunun bilgisayarda kaç saniye sürdüğü değil.”

Yoğun model

“Etkileşimli talep arttığında ilk yanıta ek olarak toplam beklemeyi de karşılaştırıyorum. İlk yanıt kazancının toplam beklemeyi iyileştirmek zorunda olmadığını bu örnekte görüyoruz.”

Gerçek Linux

“Bu bölümde gerçek süreçler çalışıyor. Önce sıfır, sonra iki arka plan işçisi kullanıyorum. P95 ve işçi bitişlerini gösteriyorum. Yeni ortamın sayıları PDF'tekilerle aynı olmak zorunda değil. Özel politika yüklenmediği için bu mevcut Linux davranışının ölçümü.”

Gerçek Linux programı çalıştırılmamışsa son paragraf canlı çalıştırılmış gibi okunmaz. Yalnız Windows modeli gösterilmişse bu kapsam açıklanır. Arşivdeki Linux kayıtları tarihsel deney olarak gösterilebilir.

Teknik soru-cevap notları

Bu Linux çekirdeğini değiştiren bir program mı? Model değiştirmez. Linux ölçümü de özel politika yüklemez; kendi süreçlerini aynı CPU'ya sabitleyip mevcut davranışı kaydeder. Çekirdek entegrasyonu sonraki aşamadır.

Neden Round Robin? Seçim sırası hesaplanıp test edilebilen anlaşılır referans. Varsayılan Linux zamanlayıcısının taklidi değil.

Neden 4, 20 ve 8 milisaniye? İlk tasarım değerleri. En iyi oldukları kanıtlanmadı; parametre duyarlılığı sonraki deneylerde incelenmeli.

Yaşlandırma kesin açlık önlüyor mu? Bütün yükler için ispat veya kesin üst sınır yok; sonlu örneklerde görev bitişi doğrulandı.

Bütçe neden katı kota değil? Diğer sınıfta hazır görev olmaması ve yaşlandırma istisnaları var. Hazır iş varken CPU boş bırakılmıyor.

CPU yönetimi ve süreç zamanlama | Hafta 2 | Sunum metni

Neden iki saat? Monoton saat bekleme/duvar süresini; süreç CPU saati CPU'da harcanan iş bütçesini ölçüyor.

P95 neyi garanti eder? Kendi örnek grubunun sıralı dağılımını özetler. Gelecekteki bütün gözlemler veya en kötü durum için garanti değildir.

CPU kullanım yüzdesi yeterli mi? Hayır. Aynı CPU kullanımında çok farklı gecikmeler olabilir; yanıt, bekleme, tamamlanma ve ilerleme ayrı izlenmeli.

13 test bütün doğruluğu kanıtlar mı? Seçilen girdileri doğrular; bütün hata koşulları, genel adalet veya çekirdek entegrasyonunu kanıtlamaz.

Model hızlı bitince gerçek politika da hızlı mı? Modelin yürütme süresiyle simüle edilen görev zamanları farklıdır. Gerçek politika maliyeti ayrıca ölçülür.

Yüzde kaç hızlandı? Genel sistem hızlanması hesaplanamaz. Belirli karışık modelde ilk yanıt 11'den 3 ms'ye inerken yoğun modelde toplam bekleme geriledi. Özel politikanın Linux üzerindeki kazancı henüz ölçülmedi.

Sıradaki kanıt ne olacak? Test VM'i ve özellik kayıtları, örnek yükleme/kaldırma, güvenli geri dönüş ve sonra gerçek politika karşılaştırma kayıtları.